Abstract

Image Geolocation and Hazard-Aware Response is a multimodal image understanding system that combines place-aware visual reasoning with risk-aware decision support. The system accepts an image from the user, analyzes visual and contextual cues using a vision-capable multimodal large language model, and returns a structured response containing summary, possible location, risk level, confidence, entities, rationale, and recommended actions. For normal landmark or place images, the system provides informational actions such as learn-more and map lookup. For hazardous or suspicious scenes, the system shows report-oriented safety actions only after explicit user confirmation.

The project is implemented as a Python Flask backend, SQLite persistence layer, and React-based web interface. Gemini 2.5 Flash is used as the primary multimodal model, while deterministic post-processing logic validates model output, normalizes confidence, applies hazard keyword rules, and maps risk states to user-facing actions. The system stores sessions, messages, model outputs, actions, and incident records for auditability. The evaluation uses project run logs containing 22 analyzed runs across 14 sessions. Low-risk and high-risk outputs each occurred eight times, uncertain outputs occurred six times, and the average confidence was 0.623. Ten user-confirmed incident records and twelve learn-more actions were logged. The results show that structured multimodal reasoning can support both geospatial understanding and controlled hazard response, while preserving human verification for safety-sensitive actions.

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The project titled **Image Geolocation and Hazard-Aware Response** is a chat-style multimodal system for image understanding, place identification, risk triage, and guided user action. The system allows a user to upload an image and receive an interpretable analysis of the scene. The response includes a short summary, possible location, detected entities, risk level, confidence score, explanation, and action buttons.

The project is developed as a web application. The frontend is implemented using React, Vite, TypeScript, Tailwind CSS, and reusable UI components. The backend is implemented using Python Flask with Pydantic validation and SQLite persistence. The primary model provider is Gemini Vision, configured to return strict JSON output. The system converts this structured model output into deterministic user actions such as *Learn More*, *Open Map*, *Safety Tips*, *Why this result?*, and *Report Suspicious Activity*.

1.2 NEED AND RELEVANCE

Images are widely shared during travel, public events, accidents, disasters, and suspicious activities. A normal image may require contextual information about a place, while a risky image may require safe reporting guidance. Traditional image geolocation systems mainly focus on predicting a place, but practical users also need risk-aware interpretation and accountable action support.

The proposed system is relevant because it joins image understanding with controlled decision support. It does not automatically notify authorities. Instead, it provides a human-in-the-loop flow where reporting is performed only after explicit user confirmation. This makes the system suitable for academic demonstration, public safety awareness, and image-based information retrieval.

1.3 MOTIVATION

The motivation behind the project is to move beyond simple image classification or geolocation and build a usable multimodal assistant. Existing visual models can

identify landmarks and describe scenes, but their responses are often free-form and difficult to audit. In safety-related scenarios, unsupported or overconfident claims can be harmful.

Therefore, the project uses structured output, validation, deterministic policy rules, and local logging. This makes the system behavior easier to explain during evaluation and safer for demonstrations involving possible hazards.

1.4 PROBLEM STATEMENT

To design and implement a multimodal web-based system that analyzes user-uploaded images, identifies possible location and scene context, detects hazard signals, and provides risk-aware actions with auditable user-confirmed reporting.

1.5 AIM AND OBJECTIVES

The aim of the project is to develop an interpretable multimodal image analysis system that combines geospatial reasoning with hazard-aware decision support.

Objectives:

- To develop a chat-style web interface for uploading and previewing images.
- To integrate a vision-capable multimodal model for structured image analysis.
- To validate and normalize model outputs into summary, risk level, confidence, entities, location, and rationale.
- To implement deterministic risk-to-action mapping for normal, uncertain, medium-risk, and high-risk scenes.
- To provide learn-more, map, safety, explanation, and user-confirmed report actions.
- To persist sessions, messages, model outputs, actions, and incidents for audit and evaluation.
- To evaluate the system using project test images and stored run logs.

1.6 PROJECT SCOPE AND LIMITATIONS

The project scope includes image upload, multimodal inference, structured response validation, confidence normalization, hazard-aware action mapping, learn-more lookup, optional map links, incident logging, and optional environment-gated WhatsApp notification after explicit confirmation.

The project does not include automatic emergency dispatch, legal identity verification, production-grade authentication, large-scale geotagged image retrieval, or forensic-level hazard verification. The current evaluation is based on a small project dataset and local run logs, so statistical conclusions are limited.

1.7 ORGANISATION OF THE REPORT

Chapter 1 introduces the project, motivation, problem statement, objectives, scope, and limitations. Chapter 2 presents the literature survey and research gap analysis. Chapter 3 describes the software requirement specification, use cases, data model, and functional model. Chapter 4 presents the project plan, estimates, risk management, and team organization. Chapter 5 explains system design, architecture, mathematical model, data design, and results. Chapter 6 presents advantages, limitations, and applications. Chapter 7 describes validation and testing. Chapter 8 concludes the report and presents future scope.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW OF EXISTING RESEARCH

Image geolocation and visual place recognition have been studied using retrieval-based methods, deep convolutional networks, and recent vision-language models. Early systems used large geotagged image collections and nearest-neighbor matching. Later approaches converted geolocation into classification over geographic regions. Recent multimodal foundation models combine image understanding with natural language reasoning, allowing richer scene interpretation and more flexible user interaction.

Hazard-aware visual decision support is related to explainable artificial intelligence, structured model outputs, safety policy design, and audit logging. In practical systems, location prediction alone is not enough. The system must also indicate uncertainty, provide rationale, and avoid automatic escalation without human confirmation.

2.2 LITERATURE SURVEY

Table 2.1: Literature Survey on Geolocation papers

Authors / Source	**Title**	**Methodology**	**Limitations**
Siński, B., Żychowski, A., Mańdziuk.	Improving Image Geolocation with Multimodal Deep Learning [1]	Image Encoder - StreetCLIP based Model - Text encoder - RoBERTa Model - self-attention mechanism	The model predicts which country an image belongs to, without finer-grained location resolution. This limits its use in applications requiring high precision
Meiliu Wu,Qunying Huang [2]	IM2City: Image Geolocalization via Multimodal Learning	Image encoder - CLIP-ViT-L/14 model Dataset - Place Plus 2.0 Model - Zero-shot GEM	Studied cities are still limited to those most populated ones in the world
Liu, L.; Han, B.; Chen, F.; Mou, C.; Xu, F [3]	Utilizing Geographical Distribution Statistical Data to Improve Zero-Shot Species Recognition	CLIP-Driven Zero-Shot Species Recognition	Data bias due to the fact that knowledge of geographic distribution comes from information incidental to the images in the dataset.

Continued on next page

Table 2.1 (continued): Literature Survey on Geolocation papers

Authors / Source	**Title**	**Methodology**	**Limitations**
Parth Parag Kulkarni, Gaurav Kumar Nayak, Mubarak Shah [4]	CityGuessr: City-Level Video Geo-Localization on a Global Scale	Self-Cross Attention module TextLabel Alignment strategy for distilling knowledge from textlabels in feature space Dataset - 'CityGuessr68k'	Limited coverage in South America and Africa
Jay, N., Nguyen, H. M., Hoang, T. D., Haimes, J [5]	Evaluating precise geolocation inference capabilities of vision language models	Vision-Language Models (O1,Gemini,Claude,Qwen)	Lack of adequate imaging in regions such as Russia, China, and Africa
Waheed, S., An, N. M., Milford, M., Ramchurn, S. D., Ehsan, S [6]	VLM-Guided Visual Place Recognition for Planet-Scale Geo-Localization	VLM + VPR-based Retrieval	VLMs alone remain unreliable as they make speculative or hallucinated guesses

Continued on next page

Table 2.1 (continued): Literature Survey on Geolocation papers

Authors / Source	**Title**	**Methodology**	**Limitations**
Lukas Haas, Michal Skreta, Silas Alberti, Chelsea Finn, et al. [7]	PIGEON/ PIGEOTTO: A Multimodal Large Language Model for Geolocalization	Multi-task contrastive pretraining, novel loss functions, and retrieval over location clusters. The model processes images by generating a latitude/longitude pair as text.	The accuracy of these models is heavily dependent on the quality and coverage of the training data. Performance can degrade in visually ambiguous areas.
Zhongliang Zhou, Jielu Zhang, Zihan Guan, Mengxuan Hu, Ni Lao, Lan Mu, Sheng Li, Gengchen Mai [8]	Img2Loc: Geolocalization as a Text Generation Task	Formulates the geolocalization problem as a text generation task, leveraging the capabilities of Large Language Models (LLMs) to predict geographic coordinates.	The approach is sensitive to the LLM's biases and can suffer from inaccuracies in regions with sparse or outdated data.
Cheng, F., Wang, J., Wang, S., Huang, Z., and Li, X. [9]	GeoGuess: Multimodal Reasoning based on Hierarchy of Visual Information in Street View	SightSense detects visual clues, retrieves multimodal geographical knowledge, and integrates both to generate context-aware visual reasoning responses.	Competetiveness in accuracy falls after other models get Knowledge along with visual clues

Continued on next page

Table 2.1 (continued): Literature Survey on Geolocation papers

Authors / Source	**Title**	**Methodology**	**Limitations**
Torneiro, A., Monteiro, D., Novais, P., Henriques, P. R., and Rodrigues, N. F [10]	Towards General Urban Monitoring with Vision-Language Models: A Review, Evaluation,and a Research Agenda	Reviewed models based on their capabilities such as vision-only,language-only and Multimodal VLMs	State-of-the-art performance in vision-language models is often achieved through extremely large and computationally expensive architectures
Li, Lingyao, Runlong Yu, Qikai Hu, Bowei Li, Min Deng, Yang Zhou, and Xiaowei Jia. [11]	From Pixels to Places: A Systematic Benchmark for Evaluating Image Geolocalization Ability in Large Language Models	IMAGEO Bench benchmarking several commercial Multimodal Models	LLMs are not yet equipped to generalize robustly across diverse global geographies

2.3 RESEARCH GAP ANALYSIS

The reviewed literature shows significant progress in image geolocation, visual place recognition, and vision-language reasoning. However, most approaches are evaluated using location accuracy, retrieval performance, or benchmark scores. In real-world use, a user may need both place information and risk-aware guidance.

The main gaps identified are:

- Lack of combined geolocation and hazard-aware response in a single user-facing workflow.
- Limited use of structured output validation for model responses.
- Limited focus on deterministic action mapping from risk level and confidence.
- Lack of human-confirmed incident reporting and audit logging in geolocation prototypes.
- Limited confidence calibration and conservative fallback design for ambiguous images.

The proposed project addresses these gaps by combining multimodal image analysis, structured JSON output, deterministic policy rules, action buttons, and persistent logs for evaluation.

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

3.1 INTRODUCTION

3.1.1 Purpose and Scope of Document

1. Facilitating other Documentation: This SRS serves as a foundational reference for all future project documentation, including system design documents, test plans, user manuals, and project reports. It ensures consistency across development phases by providing a clear understanding of system requirements and expected behavior.

2. Product Validation: The SRS enables validation by establishing measurable criteria for evaluating the system's performance and verifying that the final implementation meets user requirements. It acts as a benchmark for acceptance testing to ensure the predicted geolocation results are accurate, reliable, and aligned with project objectives.

3.1.2 Characteristics of a Software Requirement Specification

1. Accuracy: The SRS clearly defines the functional and non-functional requirements without ambiguity. Every requirement accurately reflects the intended system behavior, such as input image handling, machine learning-based geolocation prediction, and result interpretation. Accurate details ensure developers and stakeholders share a common understanding.

2. Completeness : The SRS provides a complete description of all required system features, constraints, performance expectations, and interfaces. It includes functional requirements such as dataset processing, model training, and location prediction, as well as non-functional aspects like performance, security, usability, and scalability. No essential requirement is omitted.

3. Prioritization of Requirements : The requirements in the SRS are organized based on importance and implementation priority. Critical functionalities such as image preprocessing and model prediction are placed at higher priority, followed by enhancements like user interface improvements, accuracy tuning,

and dataset expansion. Prioritization ensures efficient planning and phased development.

3.1.3 Overview of Responsibilities of Developer

The development responsibilities include frontend design, backend API implementation, multimodal model integration, database schema design, validation logic, safety policy implementation, and testing. The developer must ensure that the system does not perform automatic authority dispatch and that all report actions are explicitly confirmed by the user.

3.2 USAGE SCENARIO

The system is designed for users who want to understand an image, identify possible place context, or receive safety-aware guidance. A user opens the web application, uploads an image, and submits it for analysis. The backend sends the image and prompt to the multimodal model, validates the returned JSON, applies policy rules, saves logs, and returns a response with actions.

3.2.1 User Profiles

Table 3.1: System Entities Description

Sr. No.	Actor	Description
1	General User	Uploads an image and receives scene explanation, location estimate, risk level, and actions.
2	Public Safety Observer	Uses the report flow when a hazardous or suspicious image is detected and confirms incident logging.
3	Reviewer / Evaluator	Reviews stored model outputs, actions, confidence values, and incident records for project evaluation.
4	System Administrator	Configures API keys, database path, upload size, and optional WhatsApp notification settings.

3.2.2 Use Cases

Table 3.2: Use Cases

Sr. No.	Use Case	Description	Actors	Assumption
1	Analyze Image	User uploads an image and receives a structured scene interpretation.	User	Image format is supported.
2	Learn More	User selects learn-more action for a normal landmark or place.	User	Place/entity is detected.
3	Open Map	User opens a map search for the detected location.	User	Location guess is available.
4	View Explanation	User checks why a result was produced.	User	Rationale is returned or fallback rationale exists.
5	Report Suspicious Activity	User confirms incident reporting for high-risk output.	User	Confirmation is required before logging.
6	Review Logs	Evaluator checks sessions, actions, and incidents.	Reviewer	Database records are available.

3.2.3 Use Case View

The use case view represents interaction between users and the multimodal assistant. The primary user uploads an image and receives analysis. Based on the response, the user may learn more, open a map, view rationale, or confirm a report. The reviewer interacts with logs for evaluation.

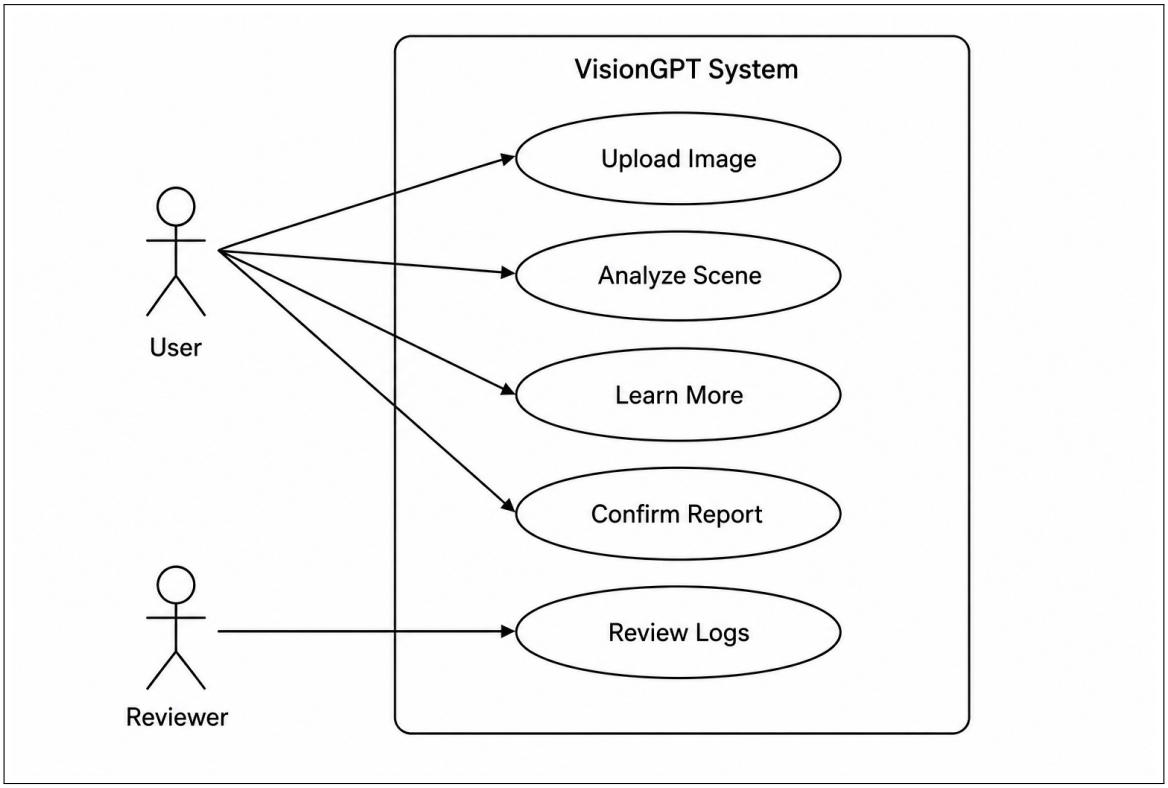

Figure 3.1: Use - Case diagram

3.3 DATA MODEL AND DESCRIPTION

3.3.1 Data Description

The system uses SQLite as the persistence layer. It stores session data, messages, structured model outputs, user actions, and incident records. Raw images are not stored by default; an image hash is saved for traceability and privacy.

3.3.2 Data Objects and Relationships

The main database tables are:

- **sessions**: Stores conversation sessions.
- **messages**: Stores user and assistant messages with optional image hash.
- **model_outputs**: Stores risk level, confidence, and full JSON payload.
- **actions**: Stores user-selected actions such as learn-more and report.
- **incidents**: Stores user-confirmed incident reports and optional notification status.

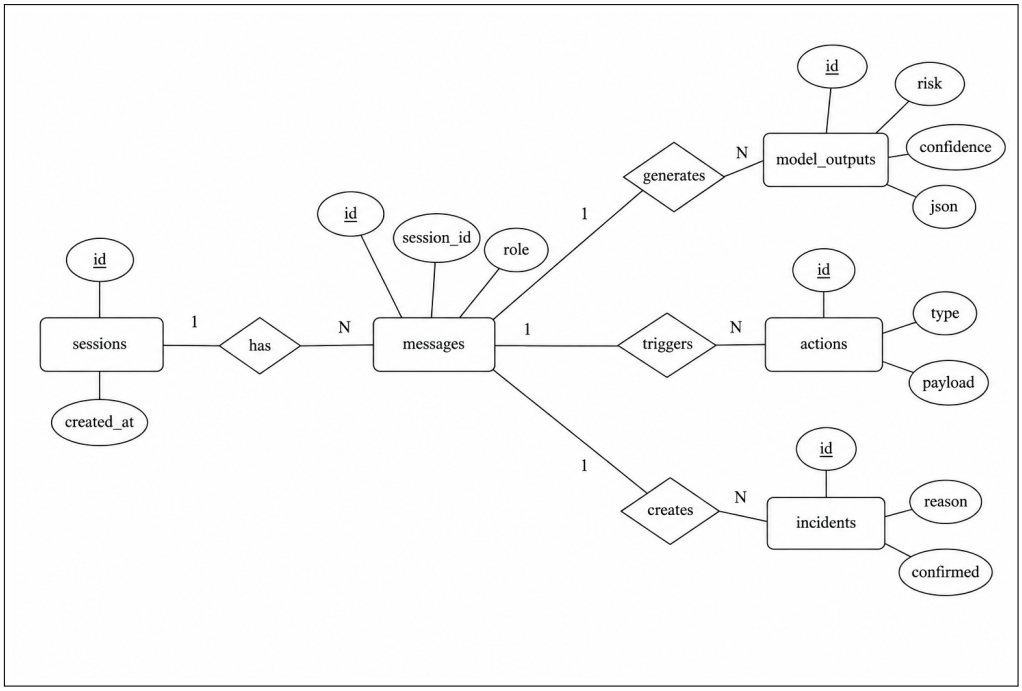

Figure 3.2: ER diagram

3.4 FUNCTIONAL MODEL AND DESCRIPTION

3.4.1 Data Flow Diagram

The data flow starts when a user submits an image. The frontend sends a multipart request to the backend. The backend validates input, hashes the image, invokes Gemini Vision, parses the model response, applies policy rules, saves records, and returns a response to the frontend.

3.4.1.1 Level 0 Data Flow Diagram

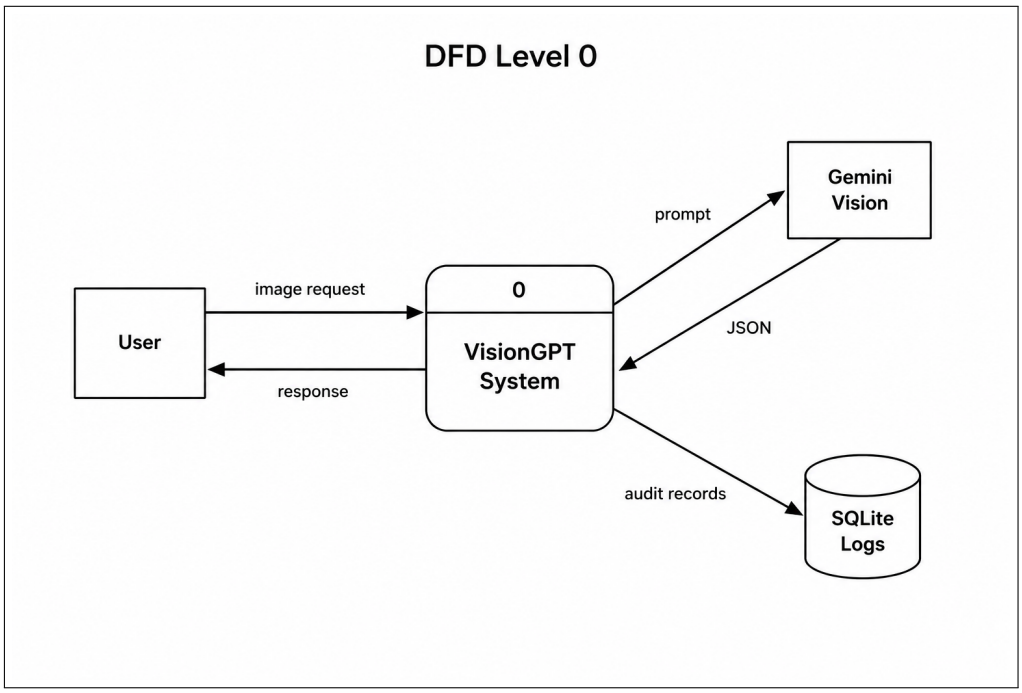

Figure 3.3: DFD0 diagram

3.4.1.2 Level 1 Data Flow Diagram

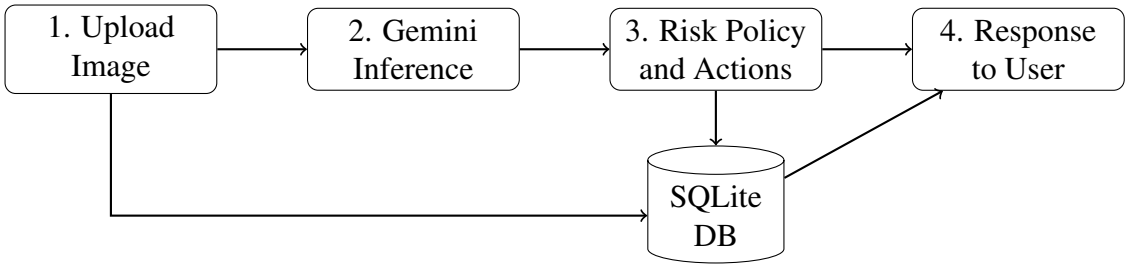

Figure 3.4: DFD Level 1 diagram

3.4.2 Description of Functions

The major functions of the system are image upload, model inference, JSON extraction, JSON repair, output normalization, hazard signal detection, action generation, learn-more lookup, map link generation, incident reporting, and audit logging.

3.4.3 Activity Diagram

The activity diagram represents the end-to-end workflow from image upload to action selection.

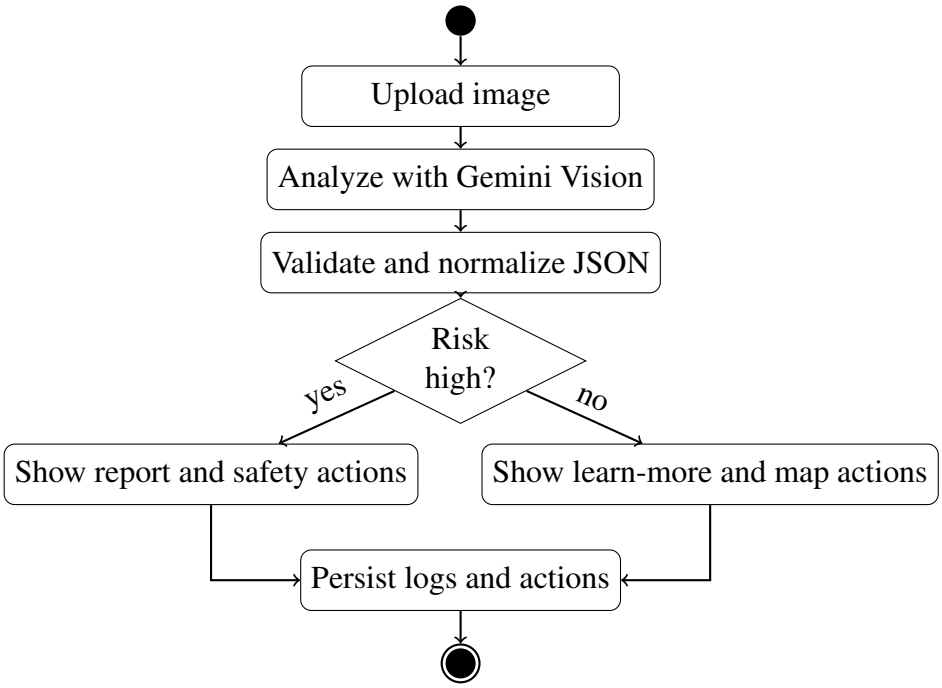

Figure 3.5: Activity diagram

3.4.4 Non Functional Requirements

- **Performance**: The MVP targets response latency below 8 seconds for common demo cases.
- **Reliability**: Invalid model output must trigger repair or safe fallback.
- **Security**: API keys must be stored in environment variables and not logged.
- **Privacy**: Raw images are not stored by default; image hashes are persisted.
- **Usability**: The interface must show image preview, chat messages, and inline action buttons.
- **Auditability**: Every model output, action, and incident must be stored for review.

3.4.5 State Diagram

The system state changes from image selection to analysis, output generation, action rendering, and optional incident creation.

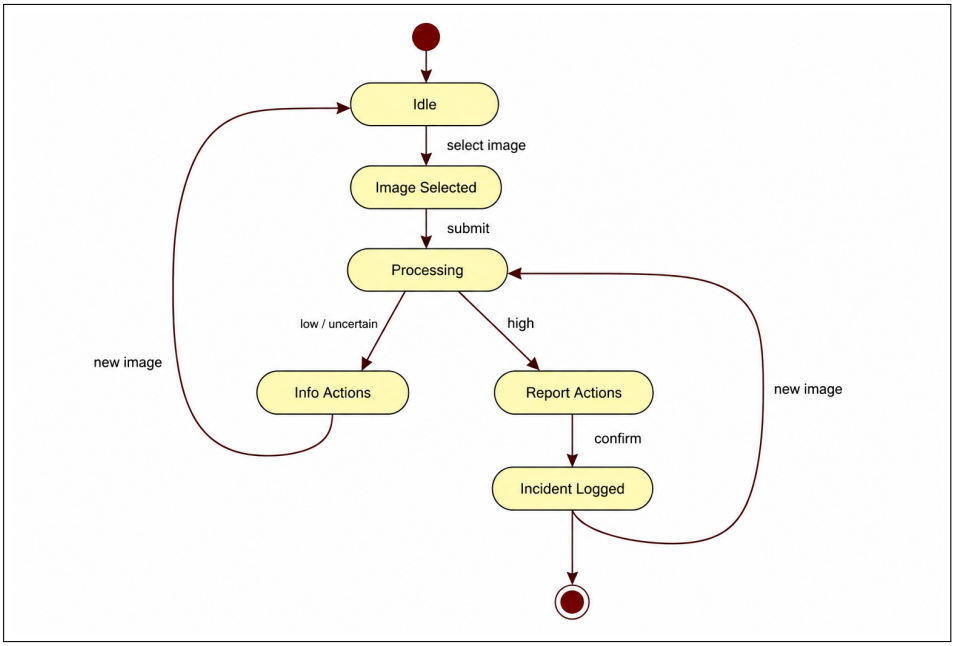

Figure 3.6: State transition diagram

3.4.6 Design Constraints

The system must not perform automatic external authority dispatch. WhatsApp notification is optional, environment-gated, and requires explicit user confirmation. The model must return strict JSON where possible, and fallback behavior must be conservative when confidence is low or output is invalid.

3.4.7 Software Interface Description

The backend exposes REST APIs for analysis, learn-more, report, and health check. The frontend communicates with these APIs using HTTP requests. Gemini Vision is accessed through the Google Generative Language API. Wikipedia is used for learn-more summaries, and Google Maps search links are generated for detected locations.

CHAPTER 4

PROJECT PLAN

4.1 PROJECT ESTIMATES

The project was planned using an iterative software development approach. The work was divided into requirement analysis, literature survey, design, implementation, integration, testing, and documentation.

1. **Requirement Gathering**: The team studied the need for image geolocation, scene explanation, risk triage, user-confirmed reporting, and audit logging.
2. **Analysis**: Functional requirements, non-functional requirements, user roles, model constraints, and safety requirements were identified.
3. **Design**: The team designed the system architecture, data model, API flow, UI flow, and risk-to-action policy.
4. **Implementation**: The backend, frontend, database schema, model integration, action handlers, and incident logging were implemented.
5. **Testing**: Normal landmark images, hazardous images, ambiguous cases, learn-more actions, and report flows were tested.
6. **Documentation**: The final report, research paper, diagrams, results, limitations, and future scope were prepared.

4.1.1 Reconciled Estimates

4.1.1.1 Cost Estimate

The project uses open-source development tools and academic infrastructure. The main cost components are internet usage, development machines, optional API usage, and documentation/printing. Since the prototype runs locally and uses SQLite, infrastructure cost is minimal.

Table 4.1: Cost Estimate

Sr. No.	Resource	Estimated Cost
1	Development laptops and college laboratory facilities	Existing resources
2	Open-source software tools	No direct cost
3	Internet and API access for model testing	Variable / demo usage
4	Report printing and binding	As per college requirement

4.1.1.2 Time Estimates

Table 4.2: Time Estimate

Sr. No.	Activity	Duration
1	Requirement analysis and literature survey	3 weeks
2	System design and database design	2 weeks
3	Backend and model integration	4 weeks
4	Frontend development and action workflow	4 weeks
5	Testing, metrics, and report preparation	3 weeks

4.1.2 Project Resources

- **People**: Four student developers, internal guide, project coordinator, and reviewers.
- **Hardware**: Development laptops with internet connection.
- **Software**: Python, Flask, SQLite, React, Vite, TypeScript, Tailwind CSS, Gemini Vision API, Wikipedia package, LaTeX.
- **Tools**: VS Code, Git, browser developer tools, draw.io/diagram tools, pdflatex.

4.2 RISK MANAGEMENT W.R.T. NP HARD ANALYSIS

The project is not an NP-hard optimization problem in the classical computational sense. However, it contains uncertainty and risk because visual scene interpretation, geolocation, and hazard detection are open-ended AI tasks. Risk management therefore focuses on model uncertainty, fallback behavior, API failure, privacy, and false alerts.

4.2.1 Risk Identification

- Model returns invalid or incomplete JSON.
- Model produces incorrect location or risk level.
- Hazard keyword fallback creates false alerts.
- API key or network failure prevents model inference.

- User reports an incident without proper context.
- Raw image storage may create privacy concerns.

4.2.2 Risk Analysis

Table 4.3: Risk Table

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Invalid model output or parsing failure	Medium	Low	High	High
2	Incorrect risk classification	Medium	Low	High	High
3	Gemini API unavailable or key missing	Low	Medium	Medium	Medium
4	False incident report by user	Low	Low	High	High
5	Privacy concern due to image handling	Low	Low	High	High

4.2.3 Overview of Risk Mitigation, Monitoring, Management

Table 4.4: Risk Mitigation Plan

Risk ID	**Mitigation Strategy**	**Monitoring Method**
1	JSON extraction, one repair pass, and safe fallback response	Check model output logs
2	Deterministic confidence thresholds and hazard policy rules	Review risk distribution and sample outputs
3	Clear backend error handling and fallback messages	Monitor API response errors
4	Require explicit user confirmation before incident creation	Review incident table
5	Store image hash instead of raw image by default	Inspect message records and configuration

4.3 PROJECT SCHEDULE

4.3.1 Project Task Set

- Task 1: Domain finalization and literature survey.

- Task 2: Requirement analysis and SRS preparation.
- Task 3: UI, API, database, and policy design.
- Task 4: Backend implementation and Gemini integration.
- Task 5: Frontend implementation and action buttons.
- Task 6: Testing with normal, hazardous, and ambiguous images.
- Task 7: Metrics analysis, report writing, and paper preparation.

4.3.2 Task Network

The task network follows a dependency chain: literature survey and requirements lead to architecture design; architecture design leads to backend and frontend implementation; implementation leads to testing; testing leads to result analysis and documentation.

4.4 TEAM ORGANIZATION

4.4.1 Team Structure

The team consists of four student members working under the guidance of the internal guide. Responsibilities were distributed across research, backend, frontend, database, testing, documentation, and presentation.

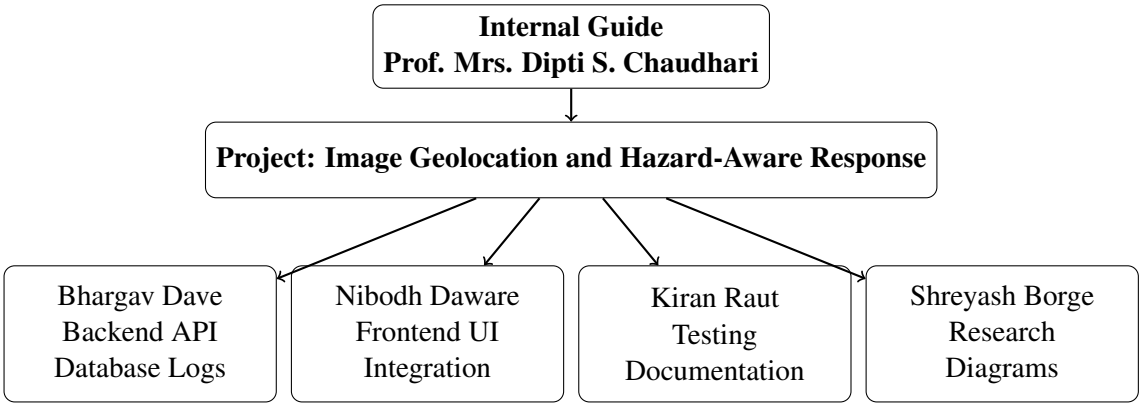

Figure 4.1: Team structure

4.4.2 Management Reporting and Communication

The team followed regular internal discussions, guide reviews, project lab sessions, and milestone-based reporting. Progress was tracked through working demonstrations, report updates, and review feedback.

Table 4.5: Management Plan

Sr. No.	Month	Description
1	June-July	Domain discussion, paper search, and problem finalization.
2	August-September	Requirement analysis, synopsis, and preliminary design.
3	October-November	Architecture, database design, and backend planning.
4	December-February	Implementation of backend, frontend, and action workflow.
5	March-April	Testing, evaluation, metrics, and report drafting.
6	May	Final report completion and submission preparation.

4.4.3 Timeline Chart

Table 4.6: Timeline Chart

Sr. No.	Task Name	Start Month	End Month
1	Domain study and base paper selection	June 2025	July 2025
2	Requirement analysis and synopsis	August 2025	September 2025
3	System design and diagrams	September 2025	October 2025
4	Backend and database implementation	November 2025	January 2026
5	Frontend and action workflow implementation	January 2026	February 2026
6	Testing, evaluation, and report preparation	February 2026	April 2026
7	Final documentation and submission	April 2026	May 2026

CHAPTER 5

SYSTEM DESIGN

5.1 INTRODUCTION

System design converts the requirements into a working architecture. The proposed system is designed as a client-server web application with a React frontend, Flask backend, SQLite database, and Gemini Vision model integration. The architecture separates user interaction, model inference, policy logic, action handling, and persistence.

5.2 SYSTEM ARCHITECTURE DESIGN

The system architecture is shown in Fig. 5.1. The frontend accepts image input and displays chat messages. The backend receives the request, validates input, sends the image to Gemini Vision, parses the response, applies normalization and risk rules, saves records, and returns action buttons to the frontend.

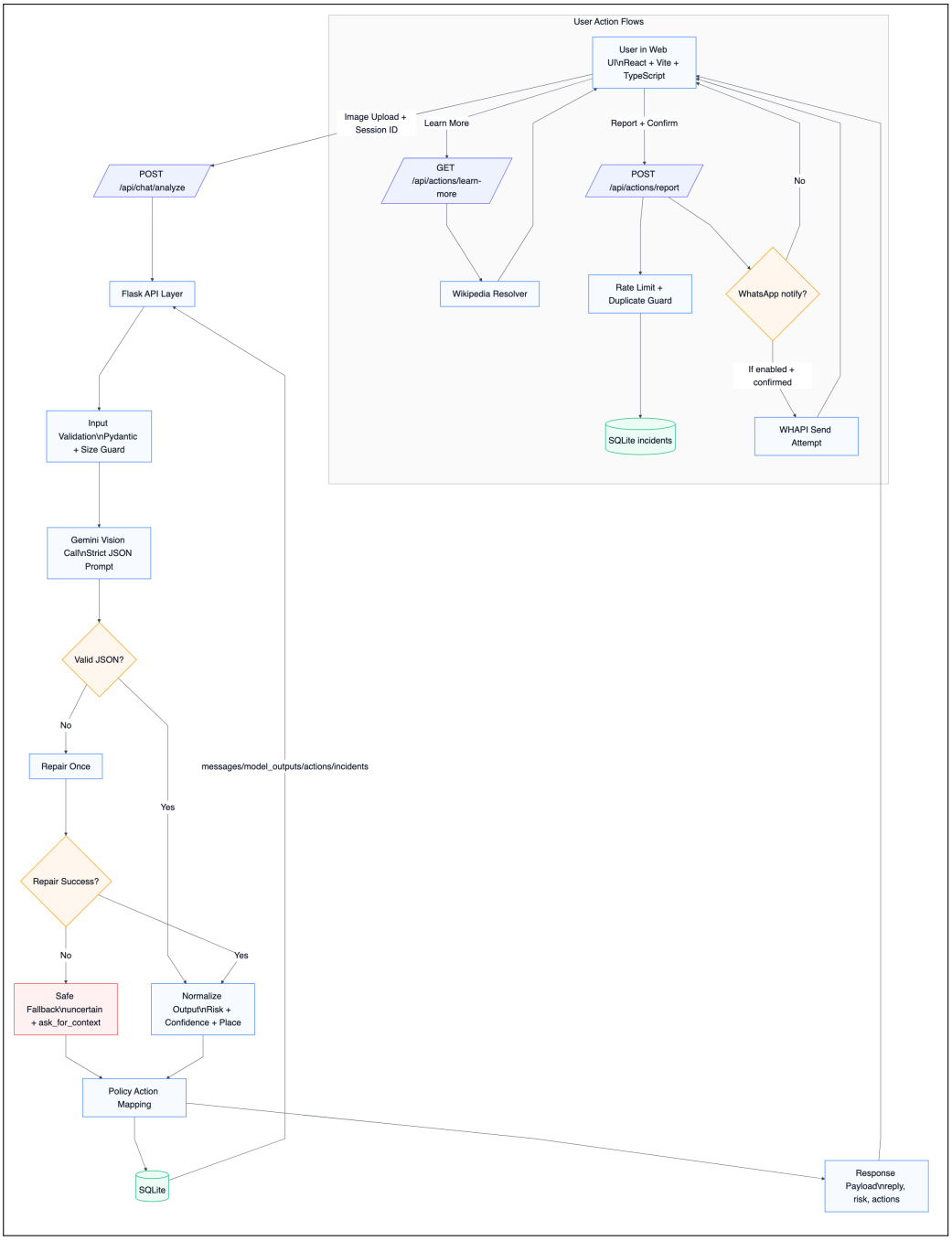

Figure 5.1: System architecture

The main modules are:

- **User Interface**: Provides image upload, preview, chat timeline, and action buttons.
- **Backend API**: Handles analyze, learn-more, report, and health endpoints.
- **Multimodal Inference**: Sends image and prompt to Gemini Vision and re-

quests strict JSON.

- **Parser and Normalizer**: Extracts JSON, repairs invalid output once, validates fields, and normalizes confidence.
- **Risk Policy**: Converts model output and hazard signals into final risk and confidence.
- **Action Mapper**: Generates actions such as report, learn-more, open map, safety tips, and why.
- **Persistence Layer**: Stores sessions, messages, model outputs, actions, and incidents in SQLite.

5.3 MATHEMATICAL MODEL FOR THE PROPOSED SYSTEM

Let the input image be represented by I and optional user context be represented by T. The multimodal model produces a structured output:

$$o = M(I, T)$$

where o contains summary, risk level, confidence, entities, possible location, rationale, and recommended actions.

Let r_m be the model risk level and c_m be the model confidence. Let $h = 1$ if hazard keywords are present in the summary, rationale, user context, or location text; otherwise $h = 0$. The final risk is:

$$r = \begin{cases} \text{high}, & h = 1 \text{ and } r_m \in \{\text{low, uncertain}\} \\ r_m, & \text{otherwise} \end{cases}$$

The confidence value is normalized to the range $[0, 1]$. When a hazard signal is detected, the confidence is adjusted as:

$$c = \max(c_m, 0.72) \quad \text{when } h = 1$$

The action set A is selected using deterministic policy rules:

$$A = \begin{cases} \{\text{ask\_for\_context}, \text{why}\}, & c < 0.45 \text{ or } r = \text{uncertain} \\ \{\text{report}, \text{safety\_tips}, \text{why}\}, & r = \text{high} \\ \{\text{safety\_tips}, \text{why}\}, & r = \text{medium} \\ \{\text{learn\_more}, \text{open\_map}, \text{why}\}, & r = \text{low} \end{cases}$$

Algorithm 1: Hazard-Aware Image Analysis

Input: Input image I and optional message T

Output: Structured response and action set A

Generate structured model output $o \leftarrow M(I,T)$;

Extract JSON from model output;

if *JSON is invalid* **then**

    Attempt one repair pass;

if *repair fails* **then**

    Return safe fallback output;

Normalize summary, risk, confidence, entities, and location;

Detect hazard signal h;

Apply risk and confidence policy;

Generate action set A;

Persist message, output, and action records;

Return response to frontend;

5.4 DATA DESIGN

The internal data structure uses Python dictionaries and Pydantic models for API validation. The global data structure is the SQLite database. Temporary data includes uploaded image bytes, base64 encoded image content, parsed model JSON, and generated action payloads.

5.4.1 Database Description

Table 5.1: Database Tables

Sr. No.	Table	Purpose
1	sessions	Stores unique user sessions and creation time.
2	messages	Stores user and assistant messages with image hash.
3	model_outputs	Stores risk level, confidence, JSON payload, and timestamp.
4	actions	Stores user action type and payload.
5	incidents	Stores confirmed reports and notification status.

5.4.2 Component Design

Table 5.2: Component Design

Sr. No.	Component	Responsibility
1	React UI	Image upload, preview, chat display, and action button rendering.
2	Flask API	Request handling, validation, response construction, and database operations.
3	Gemini Integration	Multimodal image analysis and structured output generation.
4	Policy Engine	Risk normalization, hazard keyword escalation, and action mapping.
5	SQLite Store	Persistent audit logs for sessions, messages, outputs, actions, and incidents.

5.5 RESULTS AND DISCUSSION

The project evaluation uses available run logs. A run is one invocation of the image-analysis endpoint that produces one structured model output. The available evaluation set contains 22 runs across 14 sessions.

Table 5.3: Available Dataset and Log Summary

Measure	**Value**
Total analyzed runs	22
Distinct sessions	14
Low-risk outputs	8
High-risk outputs	8
Uncertain outputs	6
Average confidence	0.623
Confidence range	0.00–0.95

Table 5.4: Hazard-Action Outcome Summary

Outcome	**Count**
User-confirmed incidents	10
Notification attempts	3
Successful notification sends	3
Report actions logged	10
Learn-more actions logged	12

(a) Regular Shaniwar Wada image (b) Hazardous Shaniwar Wada image

Figure 5.2: Test images used for case study

For the normal Shaniwar Wada image, the stored output produced low risk with confidence 0.95 and the primary action was Learn More. For the fire-affected Shaniwar Wada image, the stored output produced high risk with confidence 0.85 and the primary action was Report. This demonstrates that the same place can receive different action paths depending on visual hazard evidence.

CHAPTER 6

OTHER SPECIFICATIONS

6.1 ADVANTAGES OF THE SYSTEM

- **Multimodal understanding**: The system can analyze image content along with user context.
- **Structured output**: Model responses are converted into auditable fields instead of unstructured text only.
- **Risk-aware actions**: The response changes based on risk level and confidence.
- **Human-in-the-loop safety**: Incident reporting requires explicit confirmation from the user.
- **Audit trail**: Sessions, messages, model outputs, actions, and incident records are persisted.
- **Privacy-conscious storage**: Raw image storage is avoided by default and image hashes are stored.
- **Extensible design**: The architecture can support additional models, retrieval systems, and notification providers in future.

6.2 LIMITATIONS OF THE SYSTEM

- The evaluation dataset is small and limited to available project logs.
- The system depends on the accuracy and availability of the multimodal model.
- Hazard detection is not equivalent to legal or forensic verification.
- Confidence scores are model-provided and require stronger calibration for production use.
- The current implementation does not use vector database retrieval for geotagged image grounding.
- Hazard demonstrations are mainly fire-related, while other emergency types need more testing.

- The system does not include production-grade authentication, role-based access control, or legal compliance workflows.

6.3 APPLICATIONS OF THE SYSTEM

- Tourist landmark explanation and learn-more assistance.
- Public safety awareness and guided incident reporting.
- Academic demonstration of multimodal LLM-based decision support.
- Image-based context retrieval for educational use.
- Smart city monitoring support with human verification.
- Disaster awareness prototypes for fire, flood, collapse, or other visible hazards.
- Digital verification workflows where image context and explanation are required.

CHAPTER 7

VALIDATION AND TESTING

7.1 TESTING METHODOLOGY

Testing was performed using functional testing, integration testing, UI testing, API testing, and log-based result validation. The focus was to verify that the system accepts images, returns structured responses, handles invalid model output safely, maps risk to correct actions, and persists records.

The testing flow was:

1. Upload normal landmark image.
2. Verify low-risk response and learn-more action.
3. Upload hazardous image.
4. Verify high-risk response and report action.
5. Test ambiguous or invalid response behavior.
6. Verify database records for messages, outputs, actions, and incidents.
7. Verify report confirmation and optional notification status.

7.2 TEST CASES

Table 7.1: Test Case Summary

Sr. No	Test Case Title / Description	Testing Methodology Applied	Result	Defects Identified
1	Upload supported image file and submit for analysis	Functional testing	PASS	None
2	Analyze regular Shaniwar Wada image	System testing	PASS	None
3	Analyze fire-affected Shaniwar Wada image	System testing	PASS	None
4	Verify high-risk output shows Report action	Black-box testing	PASS	None
5	Verify low-risk output shows Learn More and Open Map actions	Black-box testing	PASS	None
6	Verify uncertain or low-confidence response asks for more context	Boundary testing	PASS	None
7	Verify report action requires confirmation before incident creation	Functional testing	PASS	None
8	Verify model output, action, and incident records are saved in SQLite	Integration testing	PASS	None
9	Verify missing/invalid model response returns safe fallback	Negative testing	PASS	None
10	Verify learn-more action fetches external knowledge summary	Integration testing	PASS	Depends on Wikipedia availability

7.3 VALIDATION OF RESULTS

The stored logs show 22 analyzed runs across 14 sessions. The risk distribution includes 8 low-risk outputs, 8 high-risk outputs, and 6 uncertain outputs. Confidence values range from 0.00 to 0.95 with an average of 0.623.

Table 7.2: Confidence Band Distribution

Band	**Count**	**Share**
Low (< 0.30)	6	27.3%
Mid (0.30–0.79)	5	22.7%
High (≥ 0.80)	11	50.0%

The case-study images validate the main intended behavior: the normal image leads to an informational action, while the hazardous image leads to a report-oriented action. The system therefore satisfies the MVP acceptance criteria for normal image interpretation, risky image escalation, ambiguity handling, and incident log review.

CHAPTER 8

CONCLUSION & FUTURE SCOPE

8.1 CONCLUSION

The project successfully demonstrates a multimodal image understanding system that combines image geolocation, scene explanation, risk triage, and guided action support. The system accepts user-uploaded images, processes them through a vision-capable multimodal model, validates structured output, applies deterministic risk policy rules, and returns user-facing actions.

The major contribution of the system is its action-oriented design. Normal place images receive informational actions such as Learn More and Open Map, while high-risk scenes receive safety-oriented actions such as Report Suspicious Activity and Safety Tips. The report flow is intentionally human-confirmed and auditable, avoiding automatic authority dispatch. This design is important because safety-related AI systems must preserve uncertainty, prevent overclaiming, and maintain accountability.

The implemented prototype uses Flask, React, SQLite, and Gemini Vision. The backend stores sessions, messages, model outputs, actions, and incident records. The frontend provides a chat-style interface with image preview and inline actions. Evaluation using stored logs shows 22 analyzed runs across 14 sessions, with low-risk and high-risk outputs each occurring eight times and uncertain outputs occurring six times. The Shaniwar Wada case study demonstrates that the same location can produce different operational outcomes depending on whether the image is normal or hazardous.

Overall, the project shows that multimodal large language models can be used not only for image description and geolocation, but also for structured and accountable decision support. The system is suitable as a proof-of-concept for academic evaluation and can be extended into a stronger safety-aware image analysis platform.

8.2 FUTURE SCOPE

- Add vector database based image retrieval for stronger geospatial grounding.
- Expand the dataset to include more cities, landmarks, weather conditions, and hazard types.

- Improve confidence calibration using labeled validation data.
- Add OCR preprocessing for clearer extraction of text from signs, boards, and scene labels.
- Add expert review workflow for confirmed incident validation.
- Add authentication and role-based access control for production deployment.
- Add dashboard analytics for risk distribution, action usage, and incident trends.
- Integrate additional notification providers while preserving explicit user confirmation.
- Support multilingual user prompts and regional emergency guidance.
- Add automated tests for parser, policy, API endpoints, and UI action flow.